DOCUMENT 02 OF 06 • TECHNICAL ARCHITECTURE SPECIFICATION

System Architecture, Flutter Client & White-Label LLM Gateway

Frontend: Flutter 3.x (Clean Architecture + Riverpod) | **Backend:** Python FastAPI + LangGraph + Celery + Typst

1. End-to-End Architectural Topology

ApexHunt AI employs a **Decoupled Hybrid Cloud-Client Architecture**. Scraping job boards, orchestrating multi-hop LLM agents, and compiling typographically crisp ATS PDFs happen on a scalable Python backend, while the Flutter mobile app acts as a responsive real-time command center, local document vault, encrypted BYOK key store, and interactive application browser.

System Tier	Core Technologies	Architectural Responsibility
Mobile Client Layer	Flutter (Dart 3), Riverpod 2.x, GoRouter, Dio, Drift (SQLite), Flutter Secure Storage	Renders Mission Control UI, streams real-time agent WebSocket logs, caches tailored PDFs and Prep Dossiers offline, encrypts user BYOK API keys, and runs assisted WebView form filling.
API & Billing Gateway	Python FastAPI, Uvicorn, Redis, RevenueCat Webhook Listener, Razorpay/Stripe	Authenticates users (Firebase/Supabase Auth), verifies active subscription status, enforces token wallet balances, and routes LLM calls via the White-Label Proxy or BYOK pass-through.
Agent Orchestration Engine	LangGraph, Celery Distributed Workers, Pydantic v2 Structured Outputs	Executes deterministic state machines: `JobScout` → `JDAnalyzer` → `ChameleonGrafter` → `ATSValidator` → `SenseiPrepBuilder`.
Discovery & Scraping Layer	Crawl4AI, Playwright Stealth, JobSpy, Upwork/Freelancer RSS & APIs, Rotating Proxies	Queries country-specific job boards and freelance marketplaces concurrently, normalizes messy HTML into clean Markdown, and filters out expired or duplicate listings.
Document Compilation & Storage	Typst CLI (Primary) + Python-Docx, Cloudflare R2 / AWS S3, PostgreSQL + pgvector	Compiles structured JSON resumes into single-column ATS-compliant PDFs in <300ms and stores versioned artifacts alongside semantic embeddings.

2. Flutter Client Architecture (Feature-First Clean Architecture)

To keep the codebase modular and ready for rapid Play Store updates (and future iOS/Web builds from the same repo), the Flutter project follows strict **Feature-First Clean Architecture** divided into `presentation`, `application` (Riverpod Notifiers), `domain` (Freezed models), and `data` (Dio repositories + Drift local DB).

```
lib/
├── core/
│   ├── network/          # Dio client, AuthInterceptor, BYOKHeaderInterceptor,
WebSocketManager
│   ├── security/         # SecureKeyVault (flutter_secure_storage wrapper for
OpenRouter/LLM keys)
│   ├── billing/          # RevenueCat & WalletBalanceController
│   └── theme/            # Material 3 Dark/Light Cyber-Executive Theme
├── features/
│   ├── master_profile/   # Base CV upload, Master Knowledge Graph viewer & lock
toggles
│   ├── mission_control/  # Country chips, Job vs. Freelance switch, Live Agent
Telemetry console
│   ├── llm_gateway/      # White-label Apex Engine selector + BYOK Custom
Provider sheet
│   ├── cv_vault/         # Side-by-side Diff Viewer, PDF previewer (pdfrx), DOCX
share sheet
│   ├── skill_dojo/       # Injected skill flashcards, STAR stories, technical Q&A
defense scripts
│   └── auto_apply/       # InAppWebView with JS injection for Greenhouse, Lever,
Workday & Upwork
└── main.dart
```

Essential Flutter Pubspec Dependencies

- **State & Routing:** `flutter_riverpod`, `riverpod_annotation`, `go_router`, `freezed_annotation`, `json_serializable`.
- **Networking & Real-Time:** `dio`, `web_socket_channel`, `eventsource` (for SSE token streaming).
- **Security & Local Persistence:** `flutter_secure_storage` (Android EncryptedSharedPreferences/Keystore), `drift` + `sqlite3_flutter_libs` (offline storage of CVs and study guides).
- **Document & Browser Utilities:** `file_picker` (CV upload), `pdfrx` (high-speed native PDF rendering), `share_plus`, `flutter_inappwebview` (semi-automated application submission).
- **Monetization:** `purchases_flutter` (RevenueCat SDK for Google Play Subscriptions & Consumable Credit Top-Ups).

3. White-Labeled OpenRouter Proxy & +$1.00 Markup Engine

The backend acts as an intelligent LLM router that completely hides OpenRouter from the frontend while supporting both high-margin managed credits and zero-cost BYOK execution.

3.1 Dynamic Model Catalog & Price Masking Pipeline

1. Every 6 hours, a background Celery beat task calls `GET https://openrouter.ai/api/v1/models`.
2. The backend selects curated models mapped to white-labeled tiers (e.g., mapping `deepseek/deepseek-chat-v3` to **Apex Turbo**, `openai/gpt-4o` to **Apex Pro**, and `anthropic/claude-3.7-sonnet` to **Apex Chameleon Elite**) and also exposes a full white-labeled model picker.
3. For every model, the backend transforms upstream pricing (which OpenRouter reports as USD per single token) by adding **+$0.000001 per token (+$1.00 per 1,000,000 tokens)** to both prompt and completion rates before sending the catalog JSON to the Flutter client.

3.2 FastAPI Token Metering & Markup Middleware (Reference Implementation)

```
from decimal import Decimal
from fastapi import Header, HTTPException

MARKUP_PER_TOKEN = Decimal("0.000001")  # Exactly $1.00 per 1,000,000 tokens
GATEWAY_BUFFER = Decimal("1.15")        # 15% buffer to absorb Google Play billing
fees

async def execute_llm_step(
    user_id: str,
    model_slug: str,
    messages: list,
    x_byok_key: str | None = Header(default=None),
    x_byok_base_url: str | None = Header(default="https://openrouter.ai/api/v1")
):
    # 1. Verify active base subscription ($3.99/mo) for all users
    wallet = await db.get_user_wallet(user_id)
    if not wallet.has_active_subscription:
        raise HTTPException(status_code=402, detail="Active base subscription
required.")

    # 2. Branch A: User supplied their own API key (BYOK Mode) -> $0 platform cost
    if x_byok_key:
        response = await call_openai_compatible(
            base_url=x_byok_base_url, api_key=x_byok_key, model=model_slug,
messages=messages
        )
        await db.log_usage(user_id, model_slug, response.usage, mode="BYOK",
charged_usd=Decimal("0"))
        return response.content

    # 3. Branch B: Managed White-Label Proxy -> Check User Wallet Balance first
    if wallet.credit_balance_usd <= Decimal("0.02"):
        raise HTTPException(status_code=402, detail="Insufficient Apex AI Credits.
Please top up.")

    # Call OpenRouter using Master Platform Key
    response = await call_openrouter_master(model=model_slug, messages=messages)
    total_tokens = Decimal(response.usage.prompt_tokens +
response.usage.completion_tokens)
    upstream_cost = Decimal(str(response.usage.total_cost_usd))

    # Apply +$1.00 per 1M tokens markup + payment processor buffer
    platform_markup = total_tokens * MARKUP_PER_TOKEN
    total_user_charge = (upstream_cost + platform_markup) * GATEWAY_BUFFER

    # Atomically deduct from user's PostgreSQL wallet & write audit ledger
    await db.deduct_wallet_and_log(
        user_id=user_id,
        model_slug=model_slug,
        prompt_tokens=response.usage.prompt_tokens,
        completion_tokens=response.usage.completion_tokens,
        upstream_cost=upstream_cost,
        markup_profit=platform_markup,
        total_charged=total_user_charge
    )
    return response.content
```

4. Job & Freelance Discovery Engine Architecture

To support both salaried job seekers and freelancers across any selected country without getting IP-blocked or draining the user's phone battery, discovery runs via a two-tier pipeline:

- **Server-Side Aggregator Connectors (Salaried Jobs):** Uses country-localized search adapters across LinkedIn Jobs, Indeed (country-specific TLDs like `indeed.co.uk`, `indeed.ae`, `in.indeed.com`), Glassdoor, Wellfound, and direct ATS feeds (Greenhouse, Lever, Ashby, Workday) filtered by `country`, `remote_status`, and `employment_type` (Full-Time / Part-Time).
- **Freelance Project Connectors (Gig Mode):** Queries Upwork RSS/API feeds, Freelancer.com Active Projects API, PeoplePerHour, Toptal public boards, and LinkedIn Posts containing hiring intent (`"looking for a freelance" AND [skill]`). Extracts client budget, hourly vs. fixed-price structure, client country, and required deliverables.
- **Semantic Pre-Filter:** Before invoking expensive LLM rewriting, the backend computes cosine similarity between the user's Master CV embedding and the scraped listings using `pgvector`, selecting the top *N* highest-potential opportunities for deep Chameleon tailoring.

5. Sub-Second ATS Document Compilation Engine (Typst + DOCX)

Many AI resume tools fail because they render HTML-to-PDF with complex CSS grids that legacy ATS parsers scramble into unreadable text. ApexHunt AI solves this deterministically:

1. The Chameleon Agent outputs strictly validated JSON adhering to our `TailoredCVSchema`.
2. The backend injects that JSON into a battle-tested, single-column **Typst** template (for PDF) and a matching **python-docx** template (for Word `.docx`).
3. Both artifacts are compiled in memory in under 250 milliseconds, uploaded to Cloudflare R2 with signed URLs, and streamed straight to the Flutter app's local SQLite cache.